

Dossier de projet

Titre RNCP Développeur Web et Mobile

Niveau 5



SHORTLINK

Sofiane Slimane
Holberton School
10/03/2025

Contexte du projet:	3
Résumé du projet:	3
Expression des besoins du projet:	3
Les contraintes du projet:	4
Les livrables attendus:	5
Gestion de projet:	6
Environnement humain:	9
Environnement technique:	10
Technologies:	10
Exemple de requête/réponse HTTP:	11
Les objectifs de qualité:	12
Réalisations côté front-end:	13
Charte graphique:	16
Schéma d'enchaînement des maquettes - Annexe 7:	17
Arborescence du site:	17
Captures d'écran de l'interface utilisateur:	18
Extrait de code d'interface utilisateur:	20
Réalisations côté back-end:	22
La base de données:	22
Script de création de la base de données:	23
Extrait de code de composant métier:	24
Extrait de code d'accès aux données:	25
Schéma du cycle de vie d'un lien court:	26
Éléments de sécurité de l'application:	28
Hachage du mot de passe:	28
Validation des requêtes HTTP:	28
Protection des routes:	29
Injection SQL:	29
Jeu d'essai:	30
Veille sur les vulnérabilités de sécurité:	31
Injection SQL:	31
Fuite d'informations sensibles:	31
Notification d'événements en lien avec l'application:	32
Mise à jour des dépendances:	32
Fixation de session:	32
Documenter le déploiement:	33
Solution d'hébergement:	33
Conclusion:	34
Annexes:	35

Contexte du projet:

Résumé du projet:

Dans le cadre de mon projet de fin d'année à Holberton School, il a été proposé à Axel NAY, également étudiant à Holberton et moi, de faire un projet de raccourcissement de liens pour l'équipe de développeurs webs du Groupe Actual. Lorsque l'utilisateur clique sur un lien raccourci, il est redirigé vers le lien associé. J'ai pu apprendre énormément en faisant ce projet, autant d'un point de vue technique que professionnel, c'est donc pour cela que j'ai décidé de vous présenter ce projet.

Expression des besoins du projet:

ShortLink a été conçu principalement pour pallier au problème des liens trop longs, en les rendant plus courts.

Le premier objectif était de concevoir une interface web permettant à des utilisateurs d'utiliser le service.

En tant qu'utilisateur, on peut utiliser ShortLink pour:

- Créer un compte pour accéder à l'interface utilisateur
- Créer des liens courts
- Désactiver des liens courts, les rendants inutilisables
- Supprimer définitivement des liens courts
- Filtrer les différents liens courts en fonction de leurs statuts (activé ou désactivé)

Le service répond également à un autre besoin, celui-ci pour les développeurs

du Groupe Actual qui souhaitent se servir de son API afin de faciliter l'intégration avec d'autres services existants.

Pour un développeur, le service permet donc de:

- D'authentifier un service à un serveur OAuth pour récupérer un token d'accès (Machine-to-Machine communication)
- Faire des requêtes API pour utiliser ShortLink à partir d'un autre service

Les contraintes du projet:

Nous avons principalement rencontré des contraintes liées au temps et à la charge de travail. Pour ce faire, nous avons réparti la charge de travail. J'ai pris la responsabilité d'apprendre les technologies côté serveur, tandis que mon coéquipier a appris les technologies côté client. La nécessité de travailler par fonctionnalité a fait que nous avons plus tard fini par apprendre les deux.

Les livrables attendus:

L'ensemble des fonctionnalités attendues par les développeurs du Groupe Actual étaient:

- Authentification OAuth pour la partie API
- Authentification pour les utilisateurs:
 - Créer un compte
- Interface web:
 - Créer un lien court
 - Consulter l'historique des liens générés

Manipuler les liens générés :

- Les liens courts peuvent être désactivés, archivés ou supprimés par l'utilisateur à tout moment
- Un lien court doit être unique
- Un lien court à une durée de vie par défaut de 60 jours, mais peut être choisis par l'utilisateur pour une durée maximale de 120 jours

Nous avons jusqu'à la fin de notre formation pour livrer le projet (Novembre 2024). Le projet nous a été donné en Juillet 2024.

Gestion de projet:

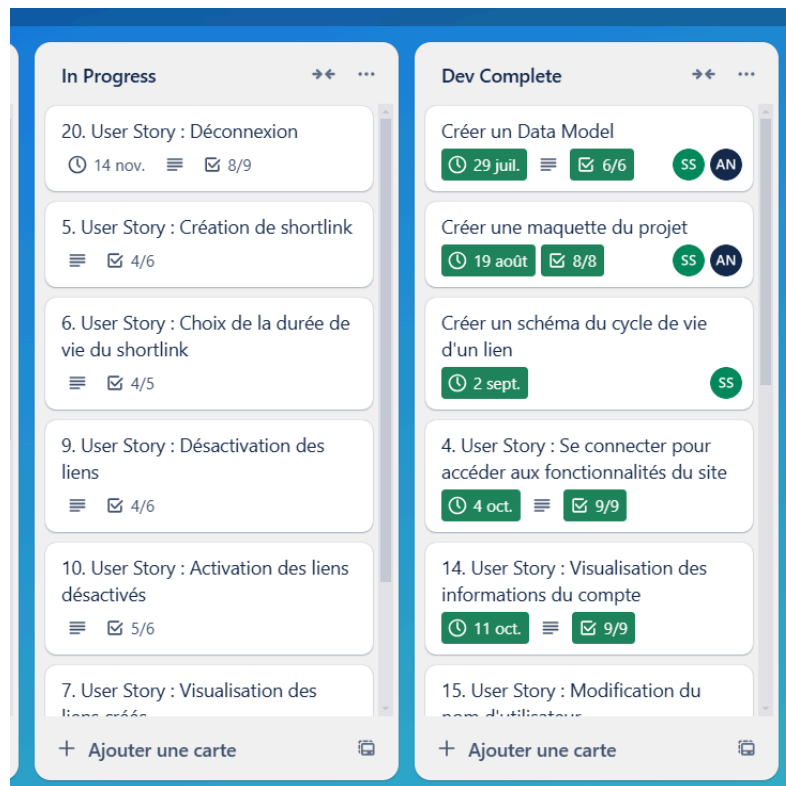
Pour la gestion du projet, nous avons décidé d'utiliser Trello.

Trello nous a permis d'identifier dans un premier temps les tâches précédents le code (création d'une maquette, d'un data model...).

Nous nous sommes ensuite servis de Trello pour diviser le projet en plusieurs "cards".

Chaque carte correspond à une fonctionnalité du projet dans laquelle on a détaillé les étapes nécessaires à la création de celle-ci.

Les cartes dont la priorité était la plus importante étaient stockées en haut d'un backlog.



Nous avons détaillé les cartes ainsi:

✓

Création de shortlink

dans la liste DEV COMPLETE ▾

Notifications

🕒 Suivre

4 oct. 2024, 12:29

Compléter ▾

🔑 Rejoindre

👤 Membres

🏷️ Étiquettes

☑️ Checklist

🕒 Dates

📎 Pièce jointe

🖼️ Image de couverture

📁 Champs personnalisés

Power-Ups

+ Ajouter des Power-ups

Automatisation ⓘ

+ Ajouter un bouton

Actions

➔ Déplacer

📄 Copier

🖼️ Miroir NOUVEAU

📄 Créer un modèle

📁 Archiver

🔑 Description

Modifier

Back-end :

- Body : url à raccourcir, durée de vie de l'url
- Code succès : 201 avec
- Code échec : 4

Front-end:

- Informations du formulaire : url, date

☑️ Back-end

Masquer les tâches cochées

Supprimer

100%

✓ Créer l'endpoint /api/link avec la méthode POST

✓ Vérifier l'utilisateur avant création du link

✓ Valider le format du lien

Ajouter un élément

☑️ Front-end

Masquer les tâches cochées

Supprimer

100%

✓ Création d'un formulaire avec les diverses informations nécessaires

✓ Requête à l'API et gestion du message de retour

Ajouter un élément

🔑 Activité

Afficher les détails

SS

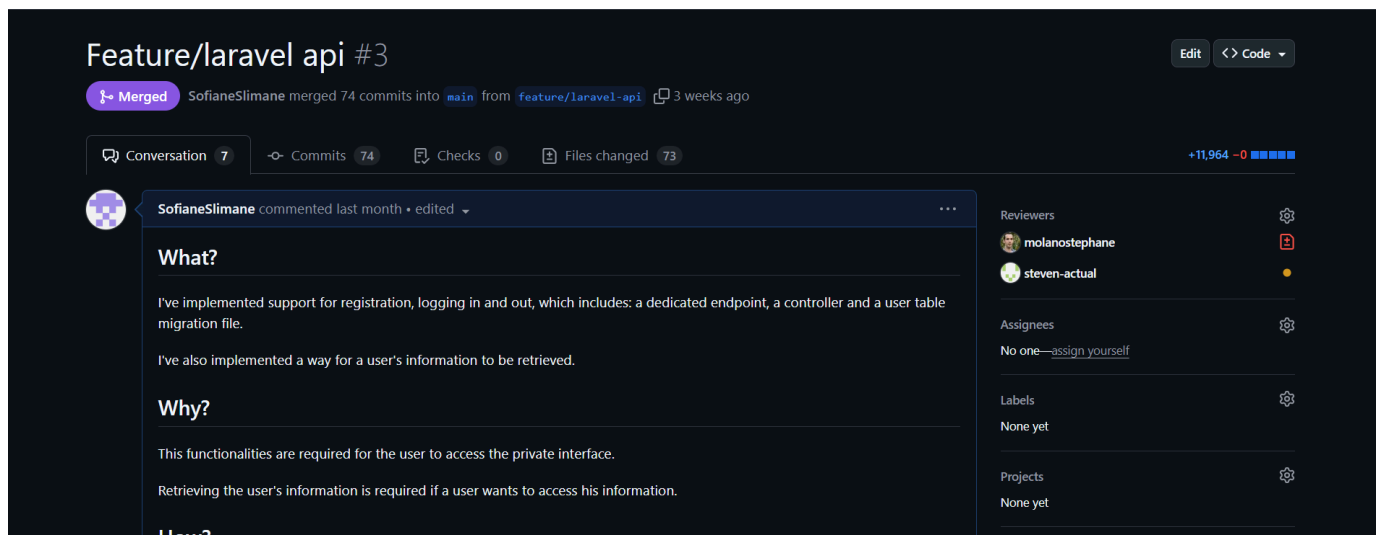
Écrivez un commentaire...

Chaque carte contient une section front-end et back-end permettant d'identifier les étapes nécessaires à l'accomplissement de la fonctionnalité.

Nous avons également fixé des dates limites afin de simuler cette idée de “sprint” et de délivrer le plus rapidement possible des fonctionnalités.

Nous nous sommes assignés certaines tâches en solo mais nous avons en grande partie accompli l'ensemble des fonctionnalités à deux. Nous avons fait beaucoup de pair programming au cours de ce projet ce qui nous a permis d'avancer et de comprendre plus rapidement l'énorme quantité de nouveaux concepts.

Nous avons ensuite fait des pull requests qui étaient passés en revue par les deux développeurs représentant du Groupe Actual, qui nous faisaient des retours.



Environnement humain:

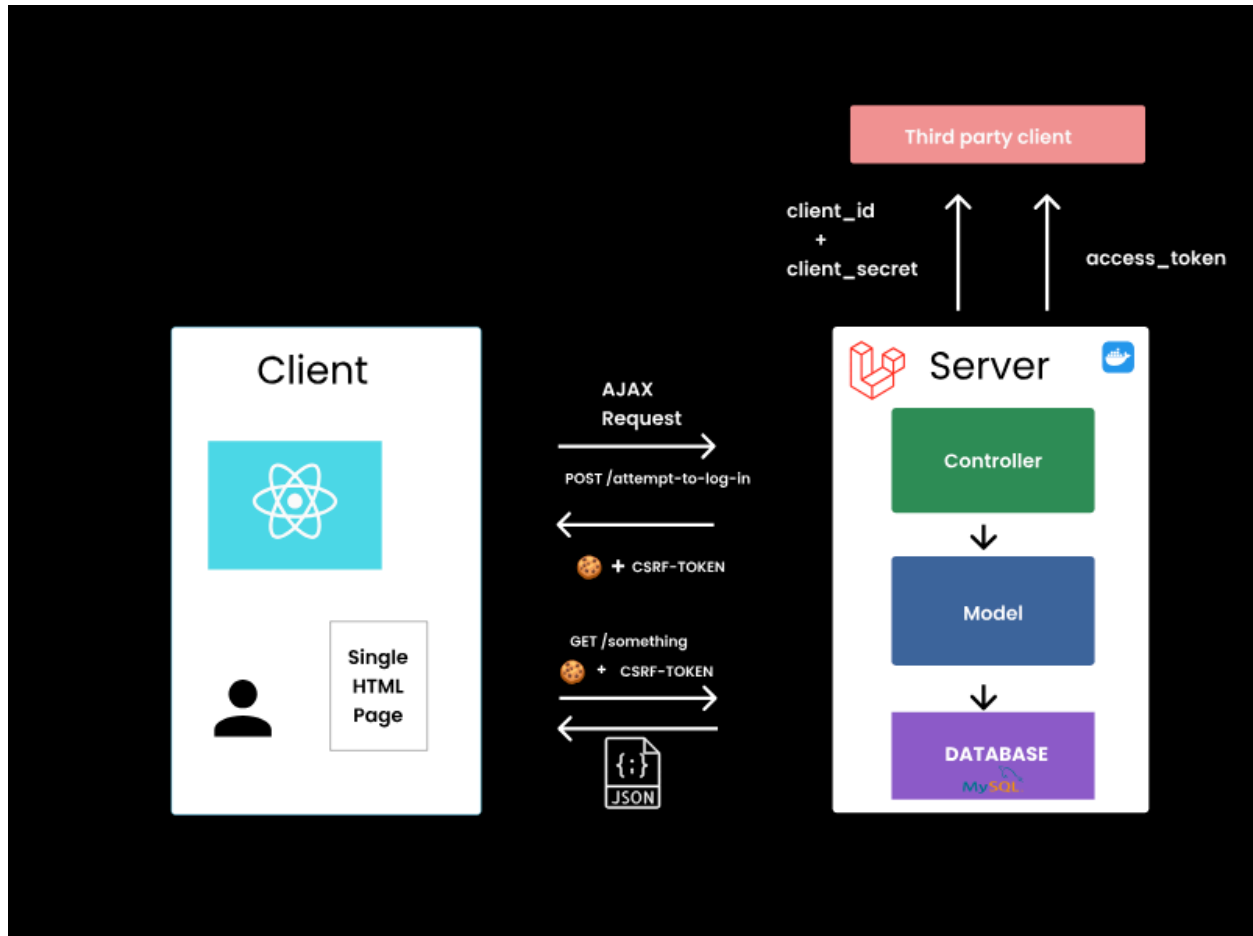
L'équipe était composée de deux étudiants d'Holberton School Bordeaux, Axel NAY et moi ainsi que de Stephane MOLANO et Steven Van POECK, deux développeurs du Groupe Actual.

Axel et moi avons communiqué principalement sur Slack lors d'échanges quotidiens, tandis que nos échanges avec Stéphane et Steven se faisaient environ une fois par mois sur Zoom. Axel et moi nous réunissions également une fois par semaine sur le campus de notre école pour faire des points plus détaillés.

Chacune de ces réunions nous a permis de redéfinir nos objectifs et de s'assurer de respecter les attendus. Le travail s'est fait en grande partie en distanciel.

Environnement technique:

Voici un schéma d'architecture de ShortLink:



Technologies:

L'ensemble des technologies utilisées étaient imposées. Pour la partie back-end nous avons utilisé le framework PHP Laravel et en front-end la librairie React pour créer des interfaces dynamiques.

Notre application Laravel et notre serveur MySQL fonctionnaient dans leurs conteneurs Docker respectifs. L'ensemble de l'application aurait dû normalement être contenu et déployé dans un (ou des) conteneur Docker mais ne l'a pas été.

React nous permet d'actualiser les parties de la représentation de notre page HTML (DOM) sans devoir actualiser l'ensemble de la page, rendant l'interface utilisateur plus rapide.

Laravel nous offre un environnement nous permettant de développer un serveur et une API très rapidement. Grâce à sa syntaxe claire et la qualité de sa documentation, Laravel est très accessible pour des développeurs juniors comme nous.

Nous nous sommes servi de git pour gérer les changements apportés au code et de Github pour l'héberger.

Exemple de requête/réponse HTTP:

Dans notre schéma, nous illustrons trois entités: le navigateur web (Client) à gauche, le serveur à droite et un service tiers qui souhaite accéder à notre API.

Dans le cas où un utilisateur normal souhaite utiliser ShortLink par l'interface, il se connecte d'abord, ce qui déclenche une requête HTTP vers le route /attempt-to-login. En cas de succès, le serveur renvoie un token CSRF que le client fournit avec les requêtes successives. Le token CSRF est une chaîne de caractère générée aléatoirement permettant de confirmer au serveur qu'une action sensible (envoi de formulaire par exemple) a bien été déclenchée par l'utilisateur et non une personne malveillante.

Dans le cas où un service tiers souhaite utiliser l'API, il doit passer par une méthode d'authentification OAuth (Client Credentials Grant). Laravel offre une implémentation d'un serveur OAuth permettant à un service tiers de fournir un client_id et un client_secret pour s'identifier. En échange, il reçoit

un token d'accès lui permettant d'accéder à l'API. Cette méthode d'authentification a été implémentée en vue d'un scénario où un service n'a pas à se soucier d'un contexte utilisateur (un id par exemple), pour lui permettre d'accéder à ses propres ressources.

Le serveur, lui, renvoie les données au format JSON soit au navigateur, dans ce cas React actualise la page avec les données reçues, où au service tiers.

Les objectifs de qualité:

- L'interface utilisateur ne doit être accessible qu'à des utilisateurs authentifiés (identifiant, mot de passe).
- l'accès à l'API par un service tiers doit aussi être restreint par une méthode d'authentification OAuth.
- Le service doit prévenir les différents types d'attaques comme les injections SQL et les attaques CSRF.
- L'ensemble de la base de données doit être formaté (Eslint et Pint) et passé en revue avant merge pour améliorer la lisibilité du code ainsi que sa qualité.
- Le choix d'une convention d'écriture doit également être fait, ici camelCase.
- Une collection regroupant les différentes routes avec leurs descriptions et des exemples de requêtes doit être fournie (Postman).

Réalisations côté front-end:

Maquettes:

Page d'accueil:

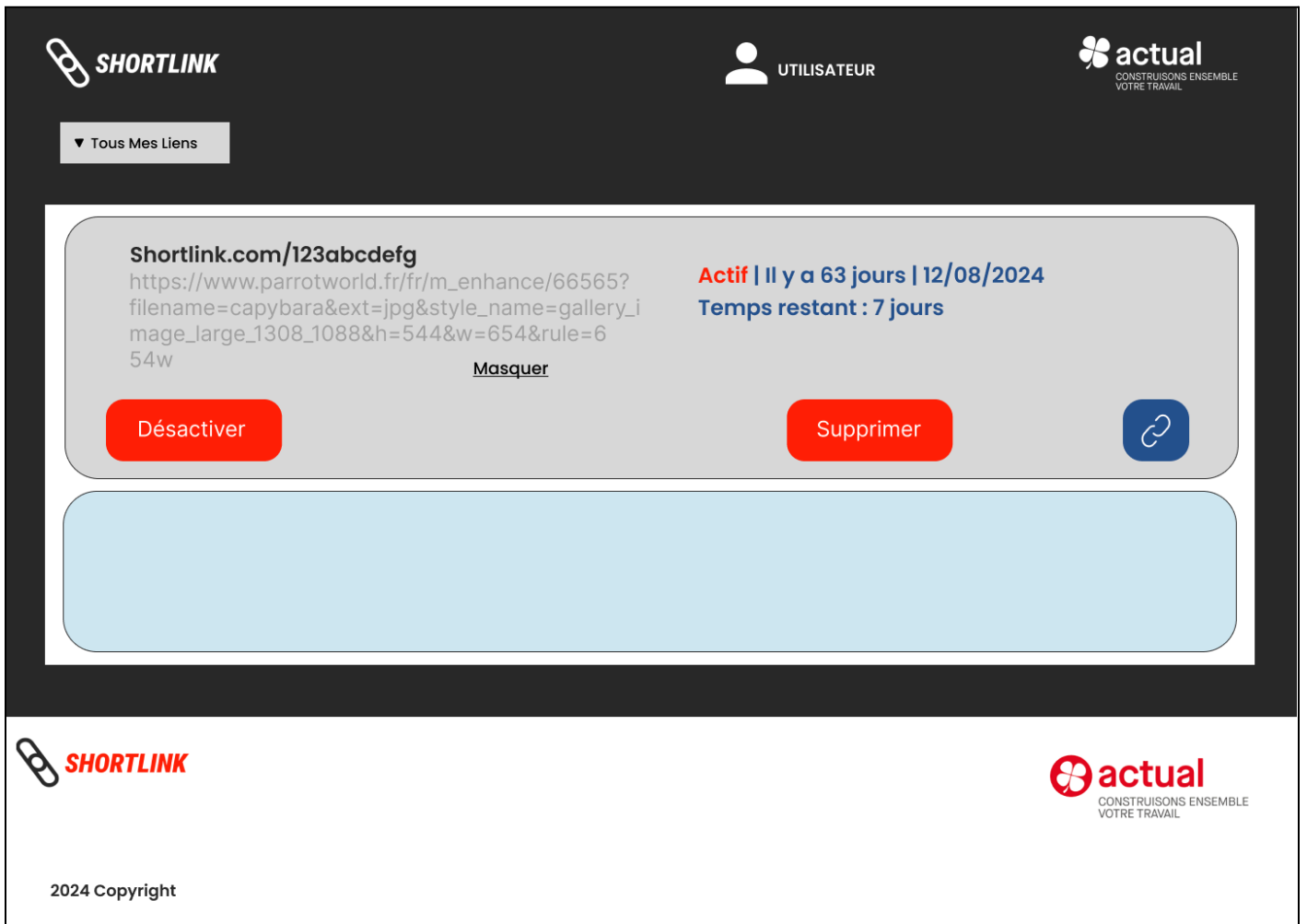


Formulaire d'inscription et de connexion - Annexe 1

Page des informations de l'utilisateur - Annexe 2

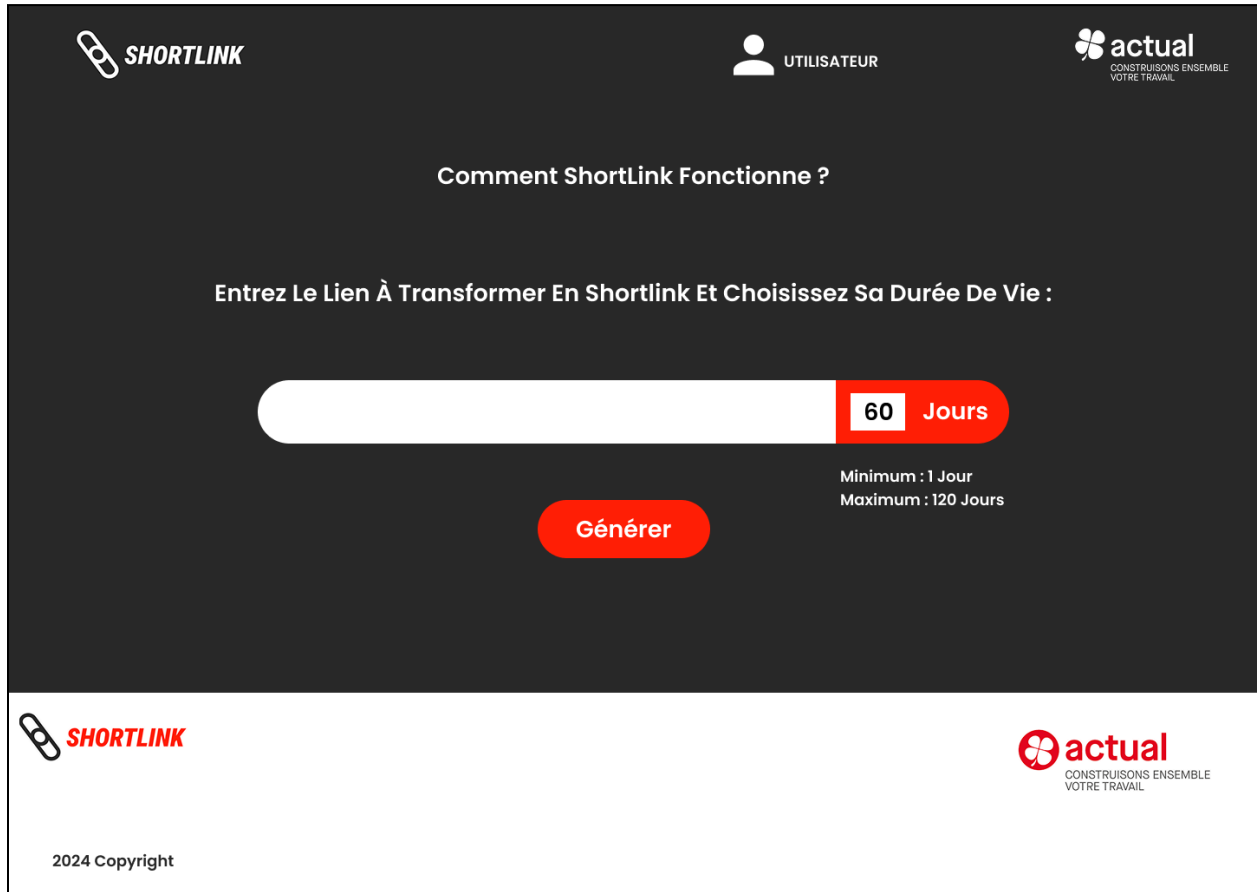
Modification des différents champs (**Annexe 3**). Nous n'avons pas gardé ces fenêtres pour le rendu final à cause de son aspect peu pratique mais il nous paraissait intéressant de les inclure.

Page d'affichage des liens courts créés:



The screenshot displays the SHORTLINK user interface. At the top, there is a dark header bar containing the SHORTLINK logo on the left, a user profile icon labeled 'UTILISATEUR' in the center, and the 'actual' logo on the right with the tagline 'CONSTRUISSONS ENSEMBLE VOTRE TRAVAIL'. Below the header, a button labeled '▼ Tous Mes Liens' is visible. The main content area shows a list of short links. The first link is 'Shortlink.com/123abcdefg', which points to a URL from 'parrotworld.fr'. It is marked as 'Actif' (Active) and was created 'Il y a 63 jours | 12/08/2024'. Below the link name, the text 'Temps restant : 7 jours' is displayed. There are three buttons below the link: 'Désactiver' (Deactivate), 'Supprimer' (Delete), and a blue circular icon with a link symbol. A 'Masquer' (Hide) link is also present. Below the first link, there is a large, empty light blue rectangular box. At the bottom of the page, there is a white footer bar containing the SHORTLINK logo on the left, the 'actual' logo on the right, and the text '2024 Copyright' on the left.

Formulaire de création d'un lien court:



The screenshot shows a web interface for creating a short link. At the top, there is a header with the 'SHORTLINK' logo on the left, a user icon labeled 'UTILISATEUR' in the center, and the 'actual' logo with the tagline 'CONSTRUISSONS ENSEMBLE VOTRE TRAVAIL' on the right. Below the header, the main content area has a dark background. It starts with the text 'Comment ShortLink Fonctionne ?'. This is followed by the instruction 'Entrez Le Lien À Transformer En Shortlink Et Choisissez Sa Durée De Vie :'. Below this instruction is a large white input field for the link. To the right of the input field is a red button with the text '60 Jours'. Below the input field and button, there is a red button labeled 'Générer'. To the right of the 'Générer' button, the text 'Minimum : 1 Jour' and 'Maximum : 120 Jours' is displayed. At the bottom of the form, there is a white footer area. On the left, it contains the 'SHORTLINK' logo and the text '2024 Copyright'. On the right, it contains the 'actual' logo and the tagline 'CONSTRUISSONS ENSEMBLE VOTRE TRAVAIL'.

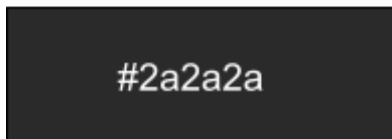
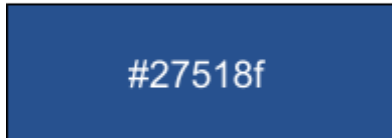
Dropdown menu permettant de filtrer les liens - Annexe 4

Page de modification du mot de passe (fonctionnalité non-complétée par manque de temps) - Annexe 5

La version finale du site n'était pas responsive mais nous avons conçu des maquettes pour une version mobile (Annexe 6).

Charte graphique:

Le site est aux couleurs du Groupe Actual (à l'exception du bleu):



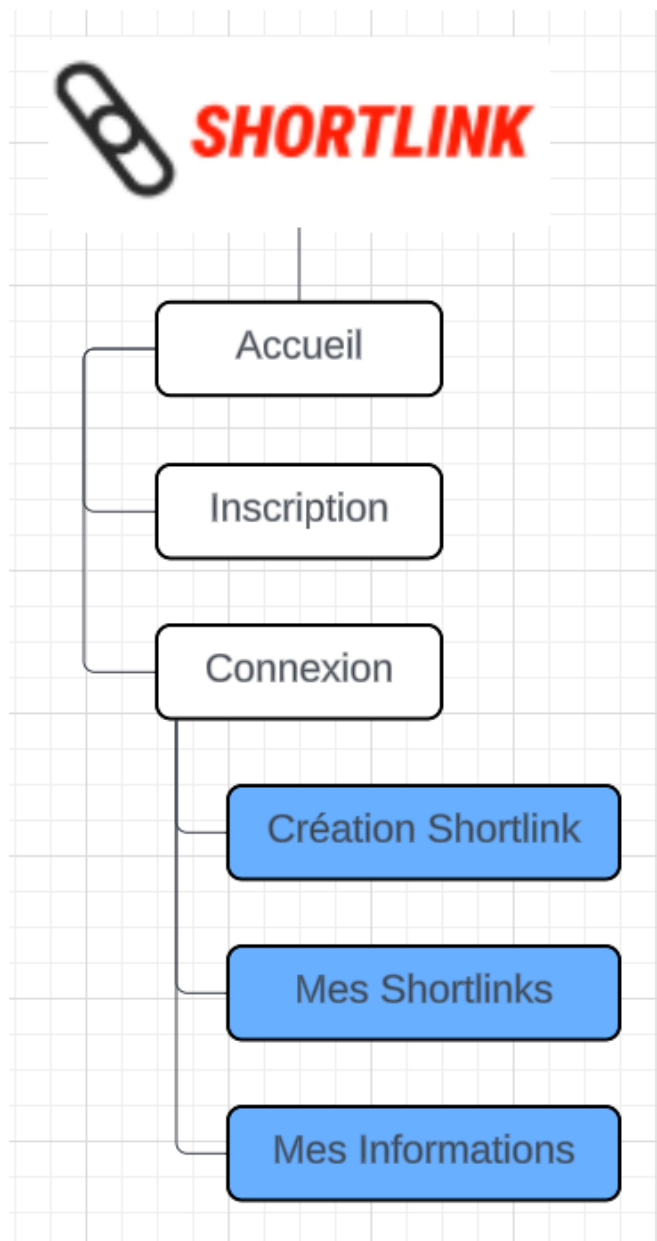
Pour la police d'écriture, nous avons choisi Poppins pour son aspect moderne.

Nous avons également choisi de commencer chaque mot par une majuscule afin d'améliorer la lisibilité.

Les deux couleurs principales du Groupe Actual étant très présentes nous avons également ajouté une nuance de bleu pour améliorer le contraste.

Schéma d'enchaînement des maquettes - Annexe 7:

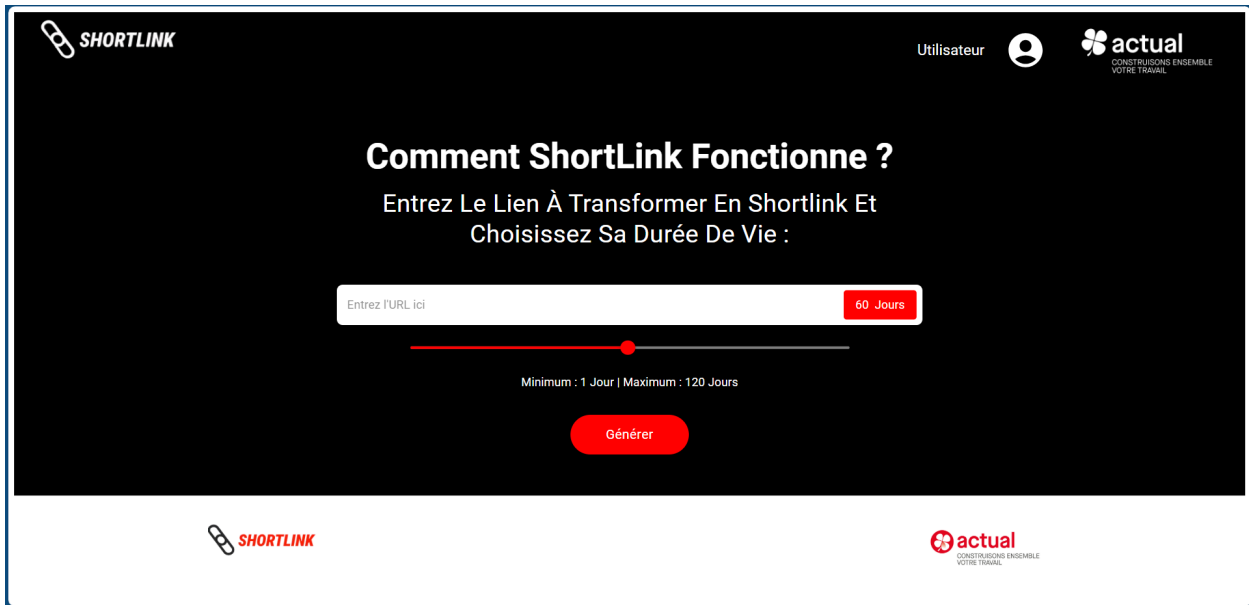
Arborescence du site:



Captures d'écran de l'interface utilisateur:

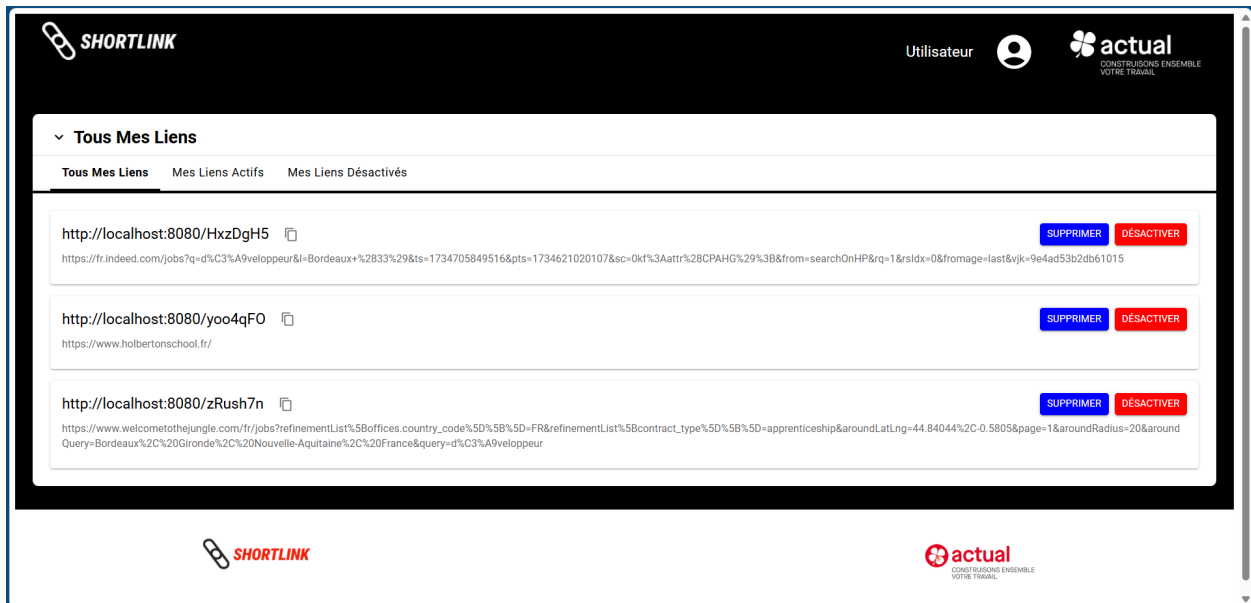
Nous avons effectué certains changements sur l'interface utilisateur qui n'étaient pas présents sur la maquette. Ce changement est notamment dû au passage sur la librairie Material UI.

Tout d'abord la page de création de liens courts:



The screenshot shows the 'Comment ShortLink Fonctionne ?' (How ShortLink works?) page. It features a dark blue header with the ShortLink logo, 'Utilisateur' (User), and 'actual' logo. The main content area has a white background with a title 'Comment ShortLink Fonctionne ?' and a subtitle 'Entrez Le Lien À Transformer En Shortlink Et Choisissez Sa Durée De Vie :'. Below this is a form with a text input field labeled 'Entrez l'URL ici' and a red button labeled '60 Jours'. A horizontal slider below the input field indicates the duration range from 'Minimum : 1 Jour' to 'Maximum : 120 Jours'. A red 'Générer' (Generate) button is at the bottom. The footer contains the ShortLink and actual logos.

Ensuite, la page qui liste les liens courts:



The screenshot shows the 'Tous Mes Liens' (All My Links) page. It features a dark blue header with the ShortLink logo, 'Utilisateur' (User), and 'actual' logo. The main content area has a white background with a title 'Tous Mes Liens' and a subtitle 'Tous Mes Liens Actifs' and 'Mes Liens Désactivés'. Below this is a list of three links, each with a 'SUPPRIMER' (Delete) button and a 'DÉSACTIVER' (Deactivate) button. The links are: 1. 'http://localhost:8080/HxzDgH5' with a copy icon and a long URL. 2. 'http://localhost:8080/yoo4qFO' with a copy icon and a URL. 3. 'http://localhost:8080/zRush7n' with a copy icon and a long URL. The footer contains the ShortLink and actual logos.

Ici, nous avons opté pour une jauge qui permet de déterminer le nombre de jours après lequel un lien court doit expirer. Lorsque l'utilisateur glisse la jauge vers la droite à l'aide de sa souris, le nombre de jours augmente. Nous avons préféré cette solution à celle où l'utilisateur entre un nombre entier car plus moderne.

Nous avons également remplacé le dropdown menu **(Annexe 4)** par trois onglets situés en haut du tableau regroupant les liens courts. Chacun de ces liens courts permet de filtrer l'ensemble des liens courts en fonction d'un critère: activé ou désactivé.

Extrait de code d'interface utilisateur:

Voici le bouton de génération de liens courts ainsi que la fonction **handleSubmit** qui gère l'événement d'envoi du formulaire.

```
<Button
  type="submit" // Submit the form
  variant="contained"
  color="third"
  sx={{
    py: 1.5, // Vertical padding for the button
    px: 6, // Horizontal padding for the button
    borderRadius: '30px', // Round the button corners
    fontSize: '1.1rem', // Font size for the button text
    textTransform: 'none', // Prevent text transformation
    fontWeight: 500, // Font weight
  }}
>
  Générer {/* Button text */}
</Button>

// Handle form submission to create a short link
const handleSubmit = async (e) => {
  e.preventDefault(); // Prevent the default form submit behavior
  try {
    const response = await createShortLink(formData); // Call the API to create the short link
    const jsonData = await response.json(); // Parse the response data

    if (response.ok) {
      setGeneratedLink(jsonData.short_link); // Set the generated short link
      setModalOpen(true); // Open the success modal
      setFormData({
        url: '', // Reset the URL field
        lifetime: 60, // Reset the lifetime field to default value
      });
    } else if (response.status === 422) {
      alert("Something is wrong with the data you sent"); // Handle invalid data error
    }
  } catch (error) {
    console.error("Error creating shortlink:", error); // Log the error in the console
    alert("An error occurred while creating the shortlink"); // Show alert if an error occurs
  }
};
```

Dans cet extrait de code, nous déclenchons la fonction **handleSumbit**. Celle-ci est asynchrone, ce qui lui permet de stopper son exécution tant que certaines actions ne sont pas terminées.

Par exemple, on tente d'envoyer une requête HTTP à l'aide de la fonction **CreateShortLink** définie dans un autre fichier. On utilise le mot clé `await` pour indiquer à **handleSumbit** de stopper son exécution tant que nous n'avons pas reçu une réponse du serveur.

Cela est aussi nécessaire pour afficher le lien court qui vient d'être créé. On récupère le lien court issu de du corps de la réponse du serveur dans la variable **jsonData**.

On vérifie ensuite le code de la réponse, "ok" si le code est dans la tranche des 200 et 422 si quelque chose ne va pas avec le format de la requête. Par exemple, un lien de redirection ne précisant pas le protocole http au début.

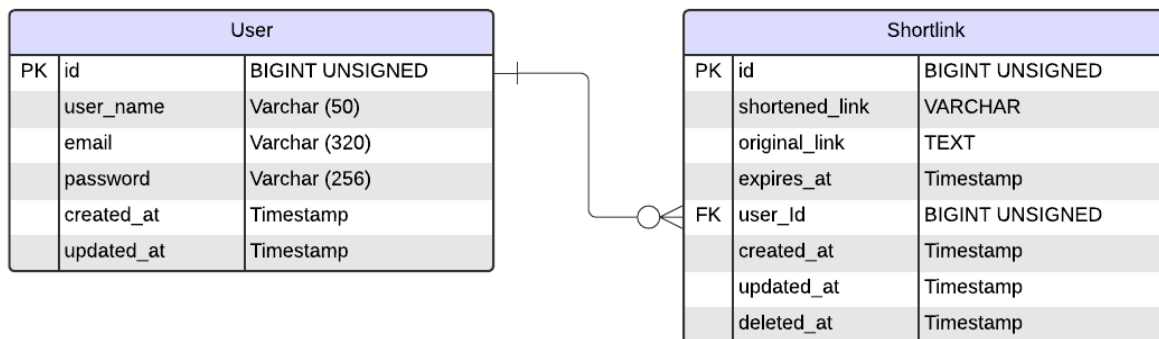
A noter que React permet de gérer l'état de certaines variables grâce au hook **useState**. Ce hook nous retourne l'état de la variable actuelle ainsi qu'une fonction permettant de le manipuler. Dans notre code, il y a deux exemples de fonction "setter", **setGeneratedLink** pour initialiser le composant affichant le lien court créé avec celui de la réponse et **setFormData** qui permet de réinitialiser le formulaire une fois le formulaire envoyé.

Réalisations côté back-end:

La base de données:

La base de données est composée principalement de deux tables (hors tables automatiquement créées par Laravel).

Une table pour les utilisateurs et une pour les liens courts.



La table User est liée à la table Shortlink par une relation one-to-many (un utilisateur peut avoir plusieurs liens courts et un lien n'appartient qu'à un utilisateur).

La liaison se fait par une foreign key dans la table Shortlink qui fait référence à la colonne id dans la table User.

La table User contient les différents champs liés à l'utilisateur.

La table ShortLink contient des informations sur le lien court comme son lien de redirection (original_link) et sa date d'expiration. On note également un champ deleted_at pour identifier les liens courts désactivés dans la base

de données (soft delete). Lorsqu'un lien est désactivé, le champ `deleted_at` est initialisé à la date de désactivation. Cela permet également à Laravel de ne récupérer que les instances qui ne sont pas désactivées lors de l'exécution de query (sauf si on le précise).

Script de création de la base de données:

Le framework Laravel fait que nous n'avons pas eu à écrire directement de script de création de base de données.

Laravel fournit une couche d'abstraction permettant à ses utilisateurs de migrer plus facilement et rapidement des tables dans la base de données.

Voici un extrait de fichier de migration sur Laravel:

```
Schema::create('sessions', function (Blueprint $table) {  
    $table->string('id')->primary();  
    $table->foreignId('user_id')->nullable()->index();  
    $table->string('ip_address', 45)->nullable();  
    $table->text('user_agent')->nullable();  
    $table->longText('payload');  
    $table->integer('last_activity')->index();  
});
```

Cet extrait de code est automatiquement généré par Laravel et permet de créer la table destinée à la gestion des sessions utilisateurs.

Laravel offre la possibilité de créer nos propres structures de tables en manipulant des objets grâce à son ORM [Eloquent](#).

Pour effectuer la migration, il suffit de taper la commande suivante dans un terminal:

php artisan migrate

Extrait de code de composant métier:

Voici un exemple de code contenant une logique métier.

```
public function register(StorePostUserRegisterRequest $request)
{
    $user = new User;
    $user->user_name = $request->user_name;
    $user->email = $request->email;
    $user->password = bcrypt($request->password);
    $user->phone_number = $request->phone_number;
    $user->save();

    return response()->json([
        'message' => 'User has successfully been created',
    ], 201);
}
```

Cette fonction est chargée d'enregistrer un utilisateur dans la base de données.

Elle prend en paramètre la requête. **StorePostUserRegister** indique à Laravel d'utiliser une classe de validation pour valider le contenu de la requête avant de la passer à la méthode **register**.

Si le contenu de la requête est validé, un utilisateur est instancié avec les champs fournis dans la requête, puis sauvegardé dans la base de données.

Après réflexion sur ce code, j'ai remarqué qu'il ne répondait pas au principe de responsabilité unique définie dans SOLID. Nous aurions pu améliorer ce code en donnant la responsabilité d'envoyer des réponses HTTP à une autre

classe. Cela aurait permis d'encapsuler un besoin métier (enregistrer un utilisateur) afin de rendre le code réutilisable.

Extrait de code d'accès aux données:

```
public function show(Authenticatable $user)
{
    $data = [
        'user_name' => $user->user_name,
        'email' => $user->email,
        'phone_number' => $user->phone_number,
    ];

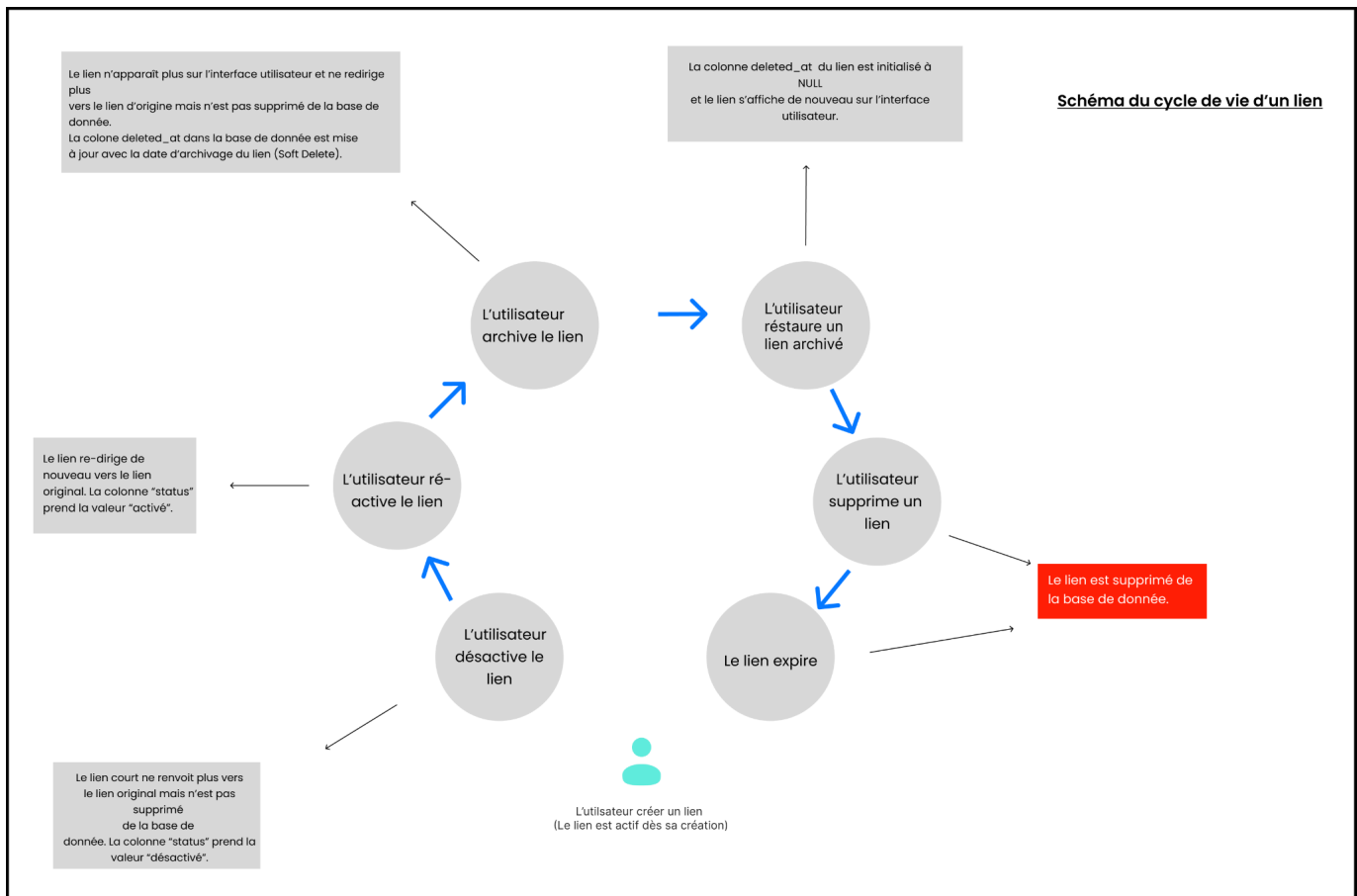
    return response()->json([
        'data' => $data,
        'status_code' => 200,
    ]);
}
```

Ici on récupère les données de l'utilisateur et on les renvoie au format JSON.

Le service container de Laravel permet d'injecter des instances de model en paramètre.

Lorsqu'on spécifie l'interface **Authenticable**, Laravel tente de trouver une instance d'un modèle qui implémente cette interface et détermine l'utilisateur actuellement connecté en se servant des informations contenues dans sa session.

Schéma du cycle de vie d'un lien court:



Lors de de l'analyse des besoins des nos clients, Axel et moi avons eu du mal à déterminer la différence entre deux fonctionnalités du service car leurs descriptions paraissent très similaires à savoir:

- Désactiver un lien court
- Archiver un lien court

Nous avons donc réalisé un schéma de cycle de vie d'un lien court pour essayer de mieux comprendre les besoins.

Nous avons imaginé une colonne **status** dans notre table qui identifie les liens désactivés et la colonne **deleted_at** pour les liens archivés.

Après avoir échangé avec Stéphane et Steven, nous en sommes arrivés à la conclusion qu'il n'y avait pas de réelle différence dans la manière dont nous envisagions de faire ces fonctionnalités. Il a donc été convenu de choisir parmi l'une des deux. Nous avons choisi d'implémenter la fonctionnalité de désactivation d'un lien court car plus intéressante pour un utilisateur.

Éléments de sécurité de l'application:

Hachage du mot de passe:

Nous avons utilisé l'algorithme de hachage bcrypt(ref. Extrait de code de composant métier) sur le mot de passe de l'utilisateur avant que celui-ci soit stocké dans la base de données.

Validation des requêtes HTTP:

Laravel nous permet de créer des “forms request”. Les forms requests sont des classes qui encapsulent la logique de validation et d'autorisation derrière une requête.

Voici un exemple de form request qui valide une requête POST, permettant la création d'un lien court (**Annexe 9**):

Cette classe permet d'abord de vérifier si l'utilisateur est autorisé à faire cette requête. La méthode **authorize** encapsule cette logique et détermine si l'utilisateur est connecté à l'aide de la méthode **check** de la façade **Auth**.

On détermine ensuite l'ensemble des règles que la requête doit respecter dans la méthode **rules**. On impose à certains champs du payload d'avoir un format en particulier:

- Le lien de redirection doit être une chaîne de caractères et préciser le protocole HTTP au début.
- Le nombre de jours après lequel le lien court expire doit être exprimé en entier.

Enfin, la méthode **failedValidation** est exécutée en cas d'échec de la validation.

Nous renvoyons une réponse avec un code 422 au client pour lui indiquer que les données contenues dans la requête sont invalides.

Protection des routes:

La plupart des routes envoyées depuis l'interface nécessite de fournir un token pour éviter les attaques CSRF.

Les routes consommées par des services tiers sont protégées par une authentification OAuth (Client Credential Grant).

Injection SQL:

Eloquent, l'ORM de Laravel utilise l'objet PDO de PHP permettant de faire des commandes SQL préparées. Les commandes SQL sont d'abord pré compilées avec des espaces réservés pour traiter l'input de l'utilisateur comme une donnée pure et non comme faisant partie de la commande SQL.

Exemple: "SELECT * FROM books where id = :id"

Ici, l'id est considéré comme une donnée et non pas comme faisant partie de la commande SQL, une tentative d'injection SQL comme "7 OR 1=1" ne fonctionnerait pas.

Jeu d'essai

Nous n'avons pas réalisé de tests lors de ce projet par manque de temps.
Cependant nous avons imaginé à quoi ces derniers pourraient ressembler:

Tests	Entrées	Sortie attendues
Accès à l'interface web par un utilisateur non connecté	Cookies et token CSRF non-fournit par le client.	Utilisateur redirigé sur la page d'accueil
Création d'un lien court	Lien de redirection, nombre de jours après lequel le lien doit expirer	Création du lien court dans la base de donnée. Code HTTP 201. Affichage d'une fenêtre avec le lien court qui vient d'être créer
Création d'un lien court avec un lien de redirection mal formaté	Lien de redirection mal formaté, ex: protocole http non précisé.	Affichage d'un message d'erreur en dessous du formulaire de création de lien courts. Code HTTP 422 pour indiquer l'invalidité des données.
Accéder à un lien court désactivé	Lien court entré dans la barre de recherche	Affichage d'une page d'erreur 404 pour indiquer que la ressource n'existe pas

Tests	Entrées	Sortie attendues
Enregistrement de l'utilisateur	nom d'utilisateur, numéro de téléphone, mot passe	Création de l'utilisateur dans la base de donnée et affichage d'un message de confirmation. Code HTTP 201.
Modification des données de l'utilisateur avec un champs déjà existant	Ex: le nom d'utilisateur JohnDoe33 est déjà utilisé.	Un message d'erreur indiquant l'existence de ce nom d'utilisateur s'affiche.e.l. You can either use the count fields to toggle t
Un tiers s'authentifie à un server OAuth avec ses coordonnées pour obtenir un token	Client id et client secret	Le service tiers obtien un token d'accès à l'API
Un tiers tente de consommer l'API sans token	Requête sans token	L'accès à l'API lui est refusé

Veille sur les vulnérabilités de sécurité

Injection SQL:

L'ORM de Laravel permet d'éviter les injections SQL en pré compilant des commandes SQL. Cependant, ce processus où l'on lie l'input de l'utilisateur à un espace réservé (ex: ":id"), pour le traiter comme une donnée pure, ne s'applique pas aux noms de colonnes. Il ne faut donc jamais laisser à l'utilisateur la possibilité de choisir le nom de la colonne référencée dans la commande SQL.

Fuite d'informations sensibles:

Il est important de prévenir la fuite d'informations sensibles liées à l'application.

Exclure certains fichiers d'un répertoire et utiliser des variables d'environnement sont des solutions à ce problème.

Pour aller plus loin, nous aurions également pu utiliser les "secrets" de Docker. Les secrets comme les mot de passe ou les tokens ont l'avantage de n'être accessibles qu'aux services auxquels nous avons donné l'autorisation dans notre fichier docker compose. A l'inverse, une variable d'environnement est accessible à l'ensemble des processus, augmentant ainsi le risque de fuite.

Notification d'événements en lien avec l'application:

Il aurait pu être envisagé d'utiliser des webhooks à l'aide de services comme Zapier pour être notifié d'un quelconque événement inhabituel en lien avec l'application. Cela permettrait d'agir plus rapidement sur d'éventuelles vulnérabilités.

Mise à jour des dépendances:

Le projet utilise beaucoup de dépendances, il est donc nécessaire de s'assurer de la mise à jour de ces dernières pour corriger d'éventuelles failles de sécurité.

Fixation de session:

Il est important de ne pas divulguer les informations de la session utilisateur. Heureusement, ces données sont stockées dans des cookies cryptés. Cela n'empêche cependant pas leurs manipulations. Un id d'une session pourrait permettre à un utilisateur malveillant de simuler la session d'un utilisateur connecté, lui permettant ainsi d'agir en son nom. Pour limiter ce risque, Laravel recommande de régénérer la session à chaque fois que l'utilisateur se connecte.

Documenter le déploiement

Solution d'hébergement

Il a été convenu avec les deux développeurs du Groupe Actual que le service, une fois terminé, aurait dû être déployé sur Google Cloud Plateforme à l'aide de nos images Docker.

Nous ne sommes pas arrivés à cette étape bien que nous ayons réussi à conteneuriser deux éléments de notre application.

J'ai donc décidé de valider cette compétence à l'aide d'un projet contenu dans mon dossier professionnel.

Conclusion:

Pour conclure, je peux dire que ce projet m'a énormément apporté en tant qu'apprentis développeur. J'ai beaucoup appris d'un point de vue technique en concevant une application complète. Les nouvelles technologies que j'ai apprises m'ont permis d'enrichir ma culture et mon opinion vis-à-vis des outils utilisés par les développeurs en entreprises. Ce que je retiens par-dessus tout, c'est l'importance que l'on doit accorder au cadre dans lequel se déroule un projet.

Effectuer l'analyse des besoins d'un client, communiquer avec celui-ci, prendre en compte ses retours, gérer un projet et travailler en équipe sont des compétences que j'ai pu développer durant cette expérience avec les membres du Groupe Actual. J'ai pu faire beaucoup d'erreurs mais j'en ressors plus confiant et prêt à appliquer ce que j'ai appris dans ma futur vie professionnelle.

Annexes:

Annexe 1:

×

Enregistrez vos informations

[J'ai Déjà Un Compte](#)

Créer Un Compte

×

Se Connecter

[Mot De Passe Oublié ?](#)

Se Connecter

[Pas Encore Inscrit ?](#) **Créer Un Compte**

Annexe 2:

 UTILISATEUR

Nom D'Utilisateur

UTILISATEUR

Numéro De Téléphone

+33123456789

Email :

utilisateur@example.com

Mot De Passe

.....

Supprimez Le Compte

Enregistrez Les Modifications



2024 Copyright

Annexe 3:

Numéro De Téléphone :

Annuler

Sauvegarder

Nom d'Utilisateur :

Annuler

Sauvegarder

Email :

Annuler

Sauvegarder

Annexe 4:

▼ Tous Mes Liens

Mes Liens Actifs

Mes Liens Désactivés

Annexe 5:



[Accueil](#) [Se Connecter](#)

Modifier Votre Mot De Passe :

Nouveau Mot De Passe :

Confirmez Votre Mot De Passe :

Modifier



2024 Copyright

[Terms of use](#) [Privacy policy](#) [Cookies policy](#)

Annexe 6:

Enregistrez Vos Informations

J'ai Déjà Un Compte

Créer un Compte

Mes Liens

Mes Informations

Se Déconnecter



Raccourcissez Vos Liens En Un Click !

Nous Rejoindre



Enregistrer Les Modifications

Supprimer Le Compte



2024 Copyright



Modifier Votre Mot De Passe :

Nouveau Mot De Passe :

Confirmez Votre Mot De Passe :

Modifier



Email :

Annuler

Sauvegarder

Annexe 7:

